AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2025

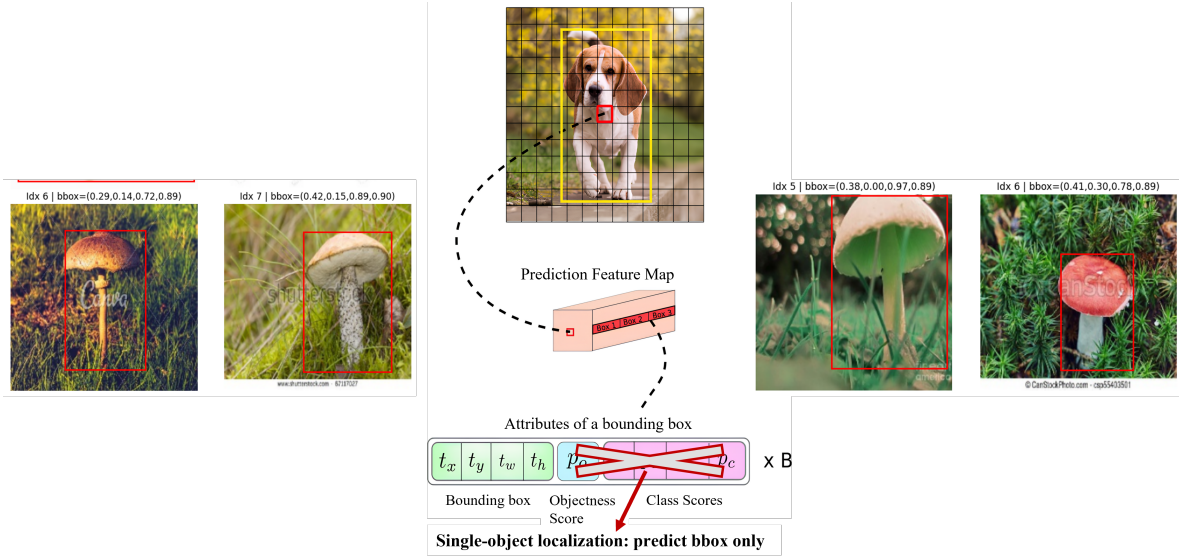
Exercise: Image Localization & POS Tagging

Nguyễn Quốc Thái Hồ Quang Hiển Đinh Quang Vinh

I. Giới thiệu

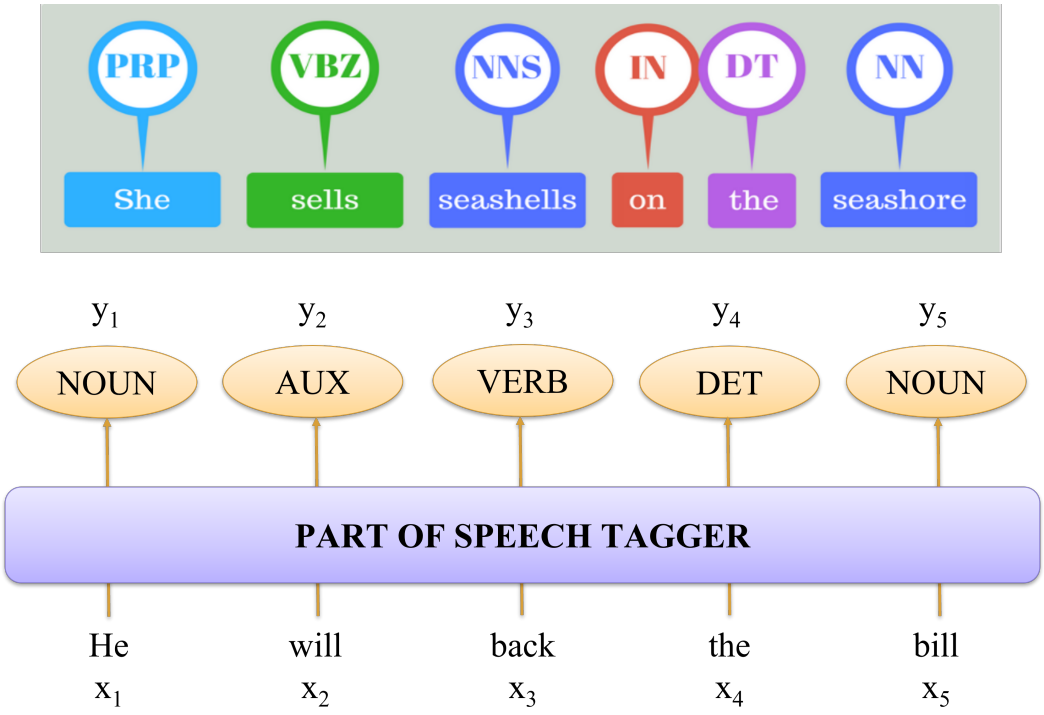
Trong thị giác máy tính hiện đại, nhiều bài toán quan trọng không dừng lại ở việc "nhận biết" (recognition) mà còn yêu cầu "định vị" (localization) chính xác trong không gian ảnh. Nếu classification trả lời *đối tượng là gì*, thì localization trả lời *đối tượng ở đâu* thông qua một khung giới hạn (bounding box). Thông tin hình học này là tiền đề để triển khai các bước suy luận tiếp theo như phát hiện nhiều đối tượng (detection), theo dõi (tracking), đếm (counting), đo kích thước ước lượng, hay điều khiển robot tương tác vật lý với môi trường.

Trong báo cáo này, chúng ta lần lượt tìm hiểu **hai bài toán nền tảng** thuộc hai miền khác nhau: (1) định vị đối tượng trong ảnh bằng hồi quy bounding box, và (2) gán nhãn từ loại trong tiếng Việt theo hướng sequence labeling. Hai bài toán đều mang tính "cấu trúc hoá" dữ liệu đầu vào: bài toán thứ nhất cấu trúc hoá ảnh bằng một biểu diễn hình học, còn bài toán thứ hai cấu trúc hoá câu bằng một lớp thông tin ngữ pháp.



Hình 1: Minh hoạ bài toán localization: dự đoán bounding box để khoanh vùng đối tượng trong ảnh.

Bài toán thứ nhất là *single-object localization* dưới dạng *bounding box regression*: với mỗi ảnh, mô hình dự đoán `(xmin, ymin, xmax, ymax)` (thường được chuẩn hoá). Một baseline phổ biến là dùng *CNN pretrained* làm *feature extractor*, rồi gắn head hồi quy nhỏ để ánh xạ đặc trưng → toạ độ hộp. Cách này gọn, dễ huấn luyện, và phù hợp khi dữ liệu hạn chế.



Hình 2: Minh hoạ POS tagging: gán nhãn từ loại cho từng token dựa trên ngữ cảnh câu.

Bài toán thứ hai thuộc xử lý ngôn ngữ tự nhiên (NLP) là *Part-of-Speech (POS) tagging*—gán nhãn từ loại cho từng token (danh từ, động từ, tính từ, ...). POS tagging cung cấp một tầng biểu diễn cú pháp cơ bản, đóng vai trò tín hiệu trung gian giúp cải thiện các tác vụ hạ nguồn như phân tích cú pháp, trích xuất thực thể (NER)...

Mục lục

I. Giới thiệu **1**

II. Bài toán Single-object Localization **4**

II.1. Định nghĩa bài toán và giả định 4

II.2. Biểu diễn bounding box và chuẩn hoá toạ độ 5

II.3. Mục tiêu học và đánh giá 5

III. Bài toán POS Tagging (Sequence Labeling) **7**

III.1. Định nghĩa bài toán (Task definition) 7

III.2. Mô hình hoá xác suất và hàm mục tiêu (Probabilistic modeling and objective) 8

III.3. Biểu diễn theo ngữ cảnh và đánh giá (Contextual representation and evaluation) 9

IV. Thực hành bài toán Single-object Localization **10**

V. Bài toán POS Tagging tiếng Việt với PhoBERT **17**

VI. Câu hỏi trắc nghiệm **25**

Phụ lục **29**

II. Bài toán Single-object Localization

Trong thị giác máy tính (computer vision), *định vị đối tượng* (object localization) là bài toán xác định vị trí của đối tượng quan tâm trong ảnh. Ở thiết lập *một đối tượng* (single-object), mục tiêu là dự đoán một *khung giới hạn* (bounding box) bao quanh đối tượng. So với *phát hiện đối tượng* (object detection) vốn cần vừa định vị vừa phân loại (classification) nhiều đối tượng, single-object localization tập trung vào thành phần hình học: dự đoán toạ độ hộp, thường dưới dạng một bài toán *hồi quy* (regression).

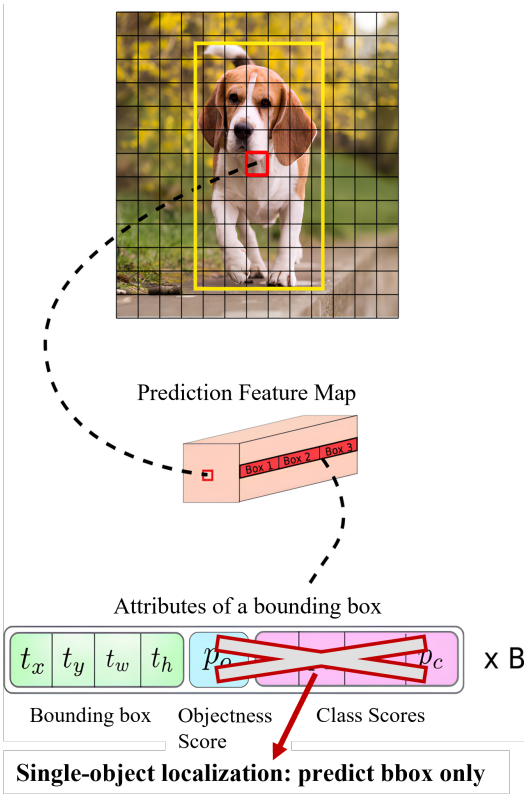
II.1. Định nghĩa bài toán và giả định

Cho ảnh đầu vào $x \in \mathbb{R}^{H \times W \times C}$, mô hình học một hàm ánh xạ tham số hoá bởi θ:

$$f_\theta : x \mapsto \hat{b},$$

trong đó $\hat{b}$ là bounding box dự đoán.

Trong thiết lập *single-object* (single-object setting), ta thường giả định mỗi ảnh có đúng một đối tượng mục tiêu hoặc ít nhất đối tượng mục tiêu luôn hiện diện. Vì vậy, mô hình có thể bỏ qua việc dự đoán *nhãn lớp* (class label) hay *điểm tin cậy* (confidence/objectness score) và chỉ tập trung vào hồi quy toạ độ hộp.



Hình 3: Mô phỏng pipeline từ ảnh tới bounding box của bài toán localization.

II.2. Biểu diễn bounding box và chuẩn hoá toạ độ

Bounding box thường được biểu diễn theo một trong hai tham số hoá phổ biến:

Dạng góc (corner form):

$$b = (x_{\min}, y_{\min}, x_{\max}, y_{\max}), \quad 0 \le x_{\min} < x_{\max} \le W, \quad 0 \le y_{\min} < y_{\max} \le H.$$

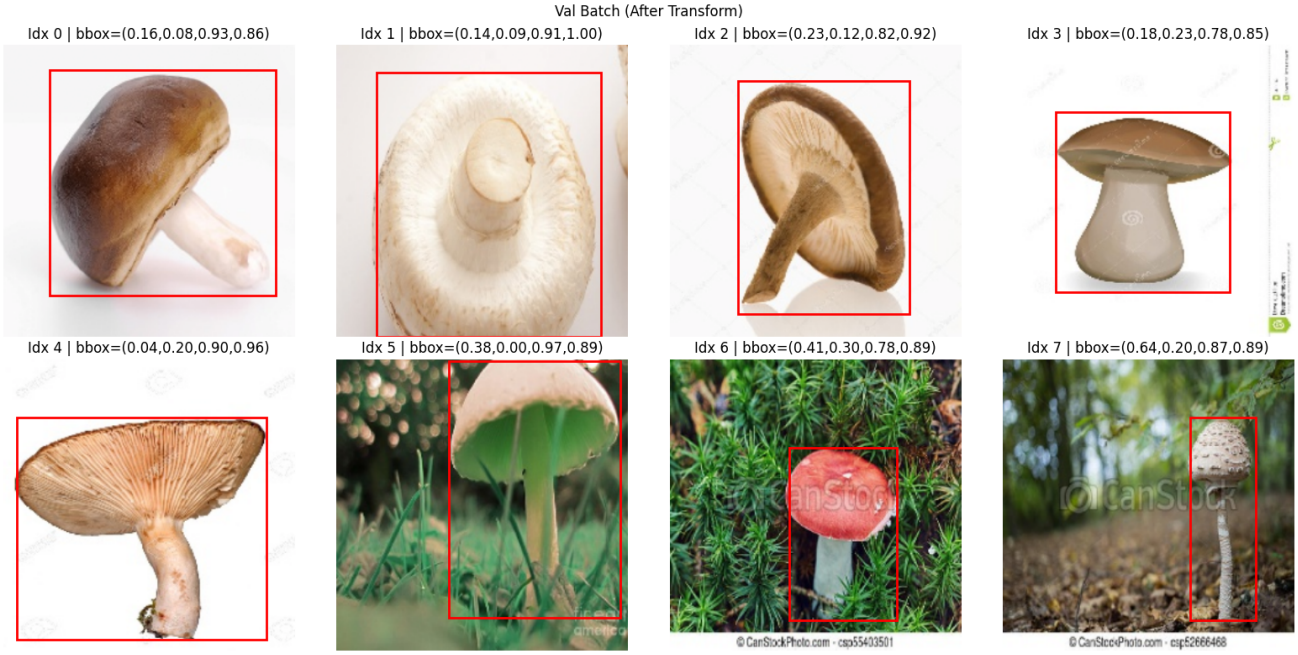
Dạng tâm–kích thước (center-size form):

$$b = (c_x, c_y, w, h), \quad c_x = \frac{x_{\min} + x_{\max}}{2}, \; c_y = \frac{y_{\min} + y_{\max}}{2}, \; w = x_{\max} - x_{\min}, \; h = y_{\max} - y_{\min}.$$

Để giảm phụ thuộc vào kích thước ảnh và giúp tối ưu hoá ổn định, toạ độ thường được *chuẩn hoá* (normalization) về miền $[0, 1]$:

$$\tilde{x} = \frac{x}{W}, \qquad \tilde{y} = \frac{y}{H}.$$

Khi đó, mô hình dự đoán $\tilde{b}$ trong không gian chuẩn hoá, và có thể chuyển ngược về toạ độ pixel khi suy luận (inference).



Hình 4: Input sau khi qua quá trình xử lý.

II.3. Mục tiêu học và đánh giá

Về bản chất, bounding box regression tối ưu tham số θ để bounding box dự đoán $\hat{b}$ gần với nhãn $b^\star$ theo một *hàm mất mát* (loss function). Một lựa chọn phổ biến là *L1 loss*:

$$\mathcal{L}_{L1}(\hat{b}, b^\star) = \|\hat{b} - b^\star\|_1,$$

hoặc *SmoothL1/Huber loss* (SmoothL1/Huber loss) nhằm tăng độ ổn định khi có ngoại lai (outliers). Với sai khác từng chiều $d = \hat{b}_i - b_i^\star$:

$$\text{SmoothL1}(d) = \begin{cases} \frac{1}{2}d^2, & |d| < 1, \\ |d| - \frac{1}{2}, & \text{ngược lại (otherwise).} \end{cases}$$

Tổng loss là:

$$\mathcal{L}_{\text{SmoothL1}}(\hat{b}, b^\star) = \sum_{i=1}^{4} \text{SmoothL1}(\hat{b}_i - b_i^\star).$$

Để đánh giá chất lượng định vị, thước đo chuẩn là *giao trên hợp* (Intersection-over-Union, IoU). Gọi A là hộp dự đoán và B là hộp ground-truth:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

IoU càng cao cho thấy hộp dự đoán càng khớp với vị trí và kích thước của đối tượng. Trong nhiều hệ thống, IoU được dùng như tiêu chí chính để so sánh mô hình, vì nó phản ánh trực tiếp mức độ chồng lấp theo hình học giữa hai bounding box.

III. Bài toán POS Tagging (Sequence Labeling)

Trong xử lý ngôn ngữ tự nhiên , *gán nhãn từ loại* (Part-of-Speech tagging, POS tagging) là bài toán gán cho mỗi đơn vị từ vựng trong câu một nhãn biểu thị vai trò ngữ pháp như danh từ, động từ, tính từ, ... POS tagging cung cấp một tầng thông tin cú pháp cơ bản, thường được dùng như tín hiệu trung gian cho các tác vụ hạ nguồn (downstream tasks) như phân tích cú pháp (parsing), trích xuất thực thể (Named Entity Recognition, NER), dịch máy (machine translation), và hệ thống hỏi–đáp (question answering).

III.1. Định nghĩa bài toán (Task definition)

Cho một câu đầu vào dưới dạng chuỗi token

$$\mathbf{x} = (x_1, x_2, \ldots, x_T),$$

mục tiêu của POS tagging là dự đoán chuỗi nhãn tương ứng

$$\mathbf{y} = (y_1, y_2, \ldots, y_T), \qquad y_t \in \mathcal{Y},$$

trong đó $\mathcal{Y}$ là tập nhãn từ loại (tag set). Đây là một dạng bài toán *gán nhãn theo chuỗi* (sequence labeling), vì mô hình cần dự đoán nhãn cho từng vị trí token trong chuỗi.

Một đặc điểm quan trọng là nhãn của một token phụ thuộc mạnh vào ngữ cảnh (context). Cùng một từ có thể nhận nhãn khác nhau tuỳ cấu trúc câu, vì vậy mô hình cần biểu diễn được phụ thuộc theo ngữ cảnh (contextual dependency) thay vì chỉ dựa trên từ độc lập.

	Tag	Description	Example
Open Class	**ADJ**	Adjective: noun modifiers describing properties	*red, young, awesome*
	ADV	Adverb: verb modifiers of time, place, manner	*very, slowly, home, yesterday*
	NOUN	words for persons, places, things, etc.	*algorithm, cat, mango, beauty*
	VERB	words for actions and processes	*draw, provide, go*
	PROPN	Proper noun: name of a person, organization, place, etc..	*Regina, IBM, Colorado*
	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	*oh, um, yes, hello*
Closed Class Words	**ADP**	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	*in, on, by under*
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	*can, may, should, are*
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	*and, or, but*
	DET	Determiner: marks noun phrase properties	*a, an, the, this*
	NUM	Numeral	*one, two, first, second*
	PART	Particle: a preposition-like form used together with a verb	*up, down, on, off, in, out, at, by*
	PRON	Pronoun: a shorthand for referring to an entity or event	*she, who, I, others*
	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	*that, which*
Other	**PUNCT**	Punctuation	; , ()
	SYM	Symbols like $ or emoji	$, %
	X	Other	asdf, qwfg

Hình 5: Các loại tag và từ tương ứng của nó.

III.2. Mô hình hoá xác suất và hàm mục tiêu (Probabilistic modeling and objective)

Cách mô hình hoá phổ biến là ước lượng phân phối có điều kiện:

$$p_\theta(\mathbf{y} \mid \mathbf{x}),$$

và tìm chuỗi nhãn có xác suất lớn nhất (maximum a posteriori, MAP):

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \; p_\theta(\mathbf{y} \mid \mathbf{x}).$$

Trong thiết lập đơn giản theo *phân loại theo token* (token-level classification), ta giả định các nhãn độc lập có điều kiện theo biểu diễn ngữ cảnh:

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{T} p_\theta(y_t \mid \mathbf{x}).$$

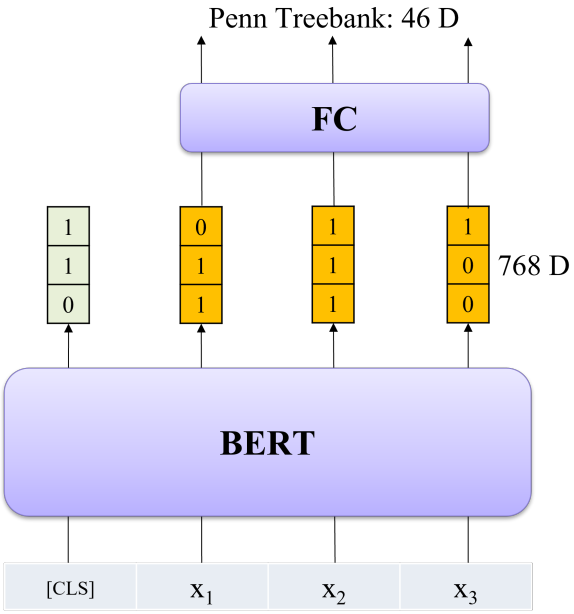
Với logits $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{Y}|}$ tại vị trí t, xác suất nhãn được tính bằng *softmax* (softmax):

$$p_\theta(y_t = k \mid \mathbf{x}) = \frac{\exp(z_{t,k})}{\sum_{j=1}^{|\mathcal{Y}|} \exp(z_{t,j})}.$$

Hàm mất mát chuẩn là *negative log-likelihood* (NLL) tương đương *cross-entropy loss* (cross-entropy loss):

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(y_t^\star \mid \mathbf{x}),$$

trong đó $y_t^\star$ là nhãn ground-truth tại vị trí t.



Hình 6: Mô phỏng pipeline Pos Tagging.

III.3. Biểu diễn theo ngữ cảnh và đánh giá (Contextual representation and evaluation)

Để giải quyết phụ thuộc ngữ cảnh (contextual dependency), các mô hình hiện đại thường dùng *mô hình ngôn ngữ tiền huấn luyện* (pretrained language model) dựa trên Transformer, nhằm tạo *biểu diễn theo ngữ cảnh* (contextualized representation) cho từng token. Gọi $\mathbf{h}_t \in \mathbb{R}^D$ là vector biểu diễn tại token t, lớp phân loại tuyến tính (linear classifier) tạo logits:

$$\mathbf{z}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b},$$

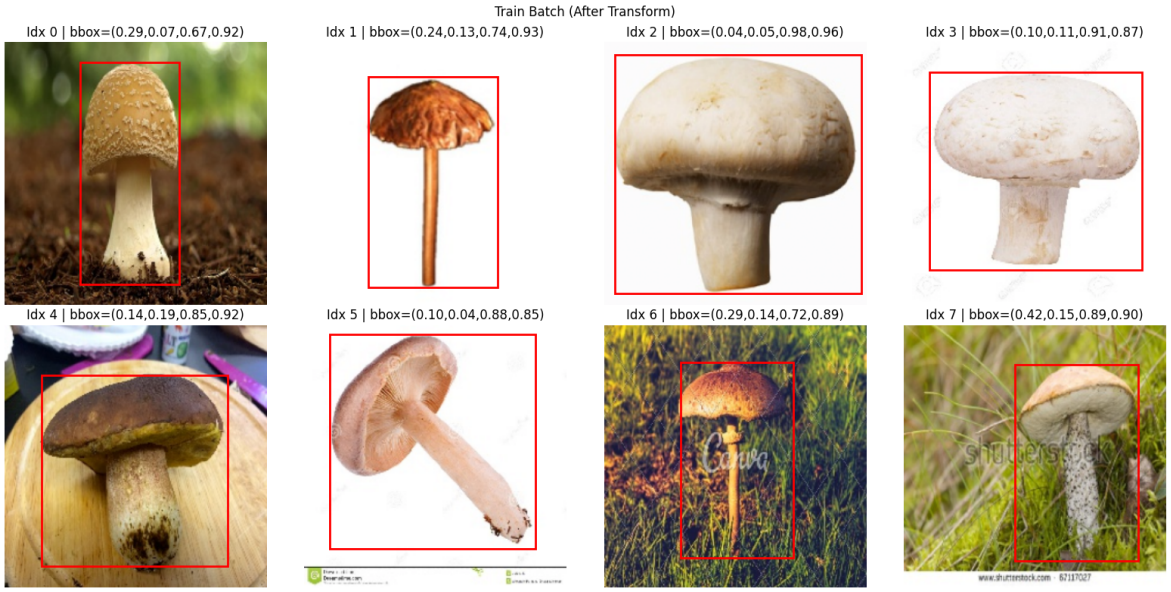
và dự đoán nhãn:

$$\hat{y}_t = \arg\max_{k \in \mathcal{Y}} \; p_\theta(y_t = k \mid \mathbf{x}).$$

Thước đo đánh giá thường dùng gồm *độ chính xác* (accuracy) theo token và/hoặc *F1-score* (F1-score) theo từng nhãn, đặc biệt khi phân bố nhãn không cân bằng (class imbalance). Trong thực tế, POS tagging được xem như một baseline quan trọng để kiểm tra mức độ mô hình hiểu cấu trúc cú pháp, đồng thời hỗ trợ tăng chất lượng cho các pipeline NLP tiếng Việt.

IV. Thực hành bài toán Single-object Localization

Trong phần này, chúng ta thực hành bài toán *định vị một đối tượng* (single-object localization) bằng cách hồi quy *khung giới hạn* (bounding box regression) cho ảnh đầu vào. Mục tiêu là dự đoán $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ bằng các *backbone CNN tiền huấn luyện* (CNN pretrained backbones) như MobileNetV3, EfficientNet và ResNet, sau đó đánh giá chất lượng định vị bằng *IoU* (Intersection-over-Union).



Hình 7: Dữ liệu của bài toán localization cho nấm.

Bước 1: Import thư viện và thiết lập cấu hình

```python
import os
import xml.etree.ElementTree as ET
from typing import List, Tuple
import numpy as np
from PIL import Image
import matplotlib.patches as patches
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models

```

```
DATA_ROOT = "mushroom_only/mushroom_only"
BATCH_SIZE = 8
NUM_EPOCHS = 15
LEARNING_RATE = 5e-4
VAL_SPLIT = 0.2
INPUT_SIZE = (320, 320)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Ở bước này, ta import các thư viện cần thiết cho bài toán *định vị một đối tượng* (single-object localization) dưới dạng *hồi quy khung giới hạn* (bounding box regression). Cụ thể, `xml.etree.ElementTree` hỗ trợ đọc nhãn từ tệp XML (Pascal VOC format), `PIL.Image` dùng để tải ảnh, và `matplotlib.patches` hỗ trợ vẽ khung giới hạn (bounding box) khi trực quan hoá (visualization). Phần huấn luyện sử dụng PyTorch (`torch`, `torch.nn`, `torch.optim`) để xây dựng mô hình (model), định nghĩa hàm mất mát (loss function) và tối ưu hoá (optimization), cùng `Dataset`/`DataLoader` để tạo mini-batch. Ngoài ra, `torchvision.models` cung cấp các backbone CNN tiền huấn luyện (pretrained CNN backbones).

Các siêu tham số (hyperparameters) chính gồm: kích thước batch `BATCH_SIZE`, số epoch `NUM_EPOCHS`, learning rate `LEARNING_RATE`, tỉ lệ chia validation `VAL_SPLIT`, kích thước ảnh đầu vào `INPUT_SIZE`, và thiết bị chạy `DEVICE`.

Bước 2: Đọc nhãn XML, chuẩn hoá bounding box, và tạo DataLoader

```
def load_image_ids(root: str) -> List[str]:
    ids = sorted({f.replace(".jpg", "") for f in os.listdir(root) if f.endswith(".jpg")
                                            })
    return ids

def parse_bbox(xml_path: str) -> Tuple[float, float, float, float]:
    tree = ET.parse(xml_path)
    root = tree.getroot()
    size = root.find("size")
    width = float(size.find("width").text)
    height = float(size.find("height").text)

    obj = root.find("object")
    bndbox = obj.find("bndbox")
    xmin = float(bndbox.find("xmin").text)
    ymin = float(bndbox.find("ymin").text)
    xmax = float(bndbox.find("xmax").text)
    ymax = float(bndbox.find("ymax").text)

    return xmin/width, ymin/height, xmax/width, ymax/height

def compute_iou(bbox1, bbox2):
    x1_min, y1_min, x1_max, y1_max = bbox1
```

```
    x2_min, y2_min, x2_max, y2_max = bbox2

    inter_min_x = max(x1_min, x2_min)
    inter_min_y = max(y1_min, y2_min)
    inter_max_x = min(x1_max, x2_max)
    inter_max_y = min(y1_max, y2_max)

    if inter_max_x < inter_min_x or inter_max_y < inter_min_y:
        return 0.0

    inter_area = (inter_max_x - inter_min_x) * (inter_max_y - inter_min_y)
    bbox1_area = (x1_max - x1_min) * (y1_max - y1_min)
    bbox2_area = (x2_max - x2_min) * (y2_max - y2_min)
    union_area = bbox1_area + bbox2_area - inter_area

    return inter_area / union_area if union_area > 0 else 0.0

class MushroomBboxDataset(Dataset):
    def __init__(self, root: str, ids: List[str], transform=None):
        self.root = root
        self.ids = ids
        self.transform = transform

    def __len__(self):
        return len(self.ids)

    def __getitem__(self, idx):
        img_id = self.ids[idx]
        img_path = os.path.join(self.root, img_id + ".jpg")
        ann_path = os.path.join(self.root, img_id + ".xml")

        image = Image.open(img_path).convert("RGB")
        bbox = torch.tensor(parse_bbox(ann_path), dtype=torch.float32)

        if self.transform:
            image = self.transform(image)

        return image, bbox

transform = transforms.Compose([
    transforms.Resize(INPUT_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

all_ids = load_image_ids(DATA_ROOT)
split_idx = int(len(all_ids) * (1 - VAL_SPLIT))
train_ids = all_ids[:split_idx]
val_ids = all_ids[split_idx:]

train_dataset = MushroomBboxDataset(DATA_ROOT, train_ids, transform=transform)
val_dataset = MushroomBboxDataset(DATA_ROOT, val_ids, transform=transform)

```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
                                                num_workers=0)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=
                                                0)
```

Ở bước này, ta xây dựng pipeline dữ liệu (data pipeline) cho bài toán bounding box regression. Nhãn bounding box được đọc từ XML (Pascal VOC format) và được *chuẩn hoá* (normalization) về miền $[0, 1]$ theo kích thước ảnh gốc:

$$\tilde{x} = \frac{x}{W}, \qquad \tilde{y} = \frac{y}{H}.$$

Cách chuẩn hoá này giúp mô hình học ổn định hơn và giảm phụ thuộc vào độ phân giải ảnh. Ảnh đầu vào được resize về `INPUT_SIZE` và chuẩn hoá theo thống kê ImageNet (ImageNet normalization) để tương thích với backbone tiền huấn luyện (pretrained backbone). Cuối cùng, `DataLoader` tạo mini-batch cho train/validation và hỗ trợ xáo trộn (shuffle) ở tập train để tăng tính ngẫu nhiên trong tối ưu hoá (optimization).

Bước 3: Khai báo mô hình (Model definition) – Bài tập điền code

Code Exercise

```
class BboxHead(nn.Module):
    def __init__(self, in_channels: int):
        super().__init__()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(in_channels, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.2),
            nn.Linear(256, 4),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = ***Your code here***
        x = ***Your code here***
        return ***Your code here***
```

Ở bài tập này, bạn cần hoàn thiện `forward` của `BboxHead` để triển khai đúng luồng *hồi quy bounding box* (bounding box regression): (1) đưa đặc trưng qua *gộp trung bình toàn cục* (global average pooling) để gom *feature map* thành vector, (2) làm phẳng tensor (flatten) theo chiều batch, và (3) đưa qua `fc` để dự đoán 4 giá trị toạ độ đã *chuẩn hoá* (normalized) trong miền $[0, 1]$ nhờ `Sigmoid`.

Code Exercise

```python
def build_model(backbone_name: str):
    backbone_name = backbone_name.lower()

    if backbone_name == "mobilenet_v3_small":
        weights = ***Your code here***
        net = ***Your code here***
        backbone = ***Your code here***
        feat_channels = 576

    elif backbone_name == "efficientnet_b0":
        weights = ***Your code here***
        net = ***Your code here***
        backbone = ***Your code here***
        feat_channels = 1280

    elif backbone_name == "resnet18":
        weights = ***Your code here***
        net = ***Your code here***
        backbone = nn.Sequential(*list(net.children())[:-2])
        feat_channels = 512

    else:
        raise ValueError(f"Unknown backbone: {backbone_name}")

    head = ***Your code here***
    model = ***Your code here***
    return model
```

Ở bài tập này, bạn cần hoàn thiện hàm `build_model` để tạo mô hình theo cấu trúc *backbone CNN tiền huấn luyện* (pretrained CNN backbone) + *đầu hồi quy* (regression head). Cụ thể, bạn cần (1) chọn đúng *trọng số tiền huấn luyện* (pretrained weights) tương ứng trong `torchvision.models`, (2) khởi tạo mạng `net` với weights đó, (3) lấy phần trích xuất đặc trưng (feature extractor) làm `backbone` cho MobileNetV3/EfficientNet (thường là `.features`), và (4) ghép `backbone` với `BboxHead(feat_channels)` để tạo mô hình dự đoán vector 4 chiều $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.

Lưu ý rằng với ResNet18, phần `backbone` đã được cố định bằng cách bỏ hai lớp cuối để giữ lại *feature map* 2D; vì vậy các vị trí cần điền trong nhánh này chủ yếu là *weights* (weights) và khởi tạo `net`. Sau khi hoàn thiện, mô hình sẽ phù hợp với thiết lập *single-object localization* (single-object localization) trong đó đầu ra chỉ là bounding box đã chuẩn hoá, không bao gồm *điểm lớp* (class scores) hay *độ tin cậy* (objectness/confidence).

Bước 4: Huấn luyện và đánh giá (Loss + IoU)

```python
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0

    for imgs, bboxes in loader:
        imgs = imgs.to(device)
        bboxes = bboxes.to(device)

        preds = model(imgs)
        loss = criterion(preds, bboxes)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * imgs.size(0)

    return total_loss / len(loader.dataset)

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    total_iou = 0.0
    count = 0

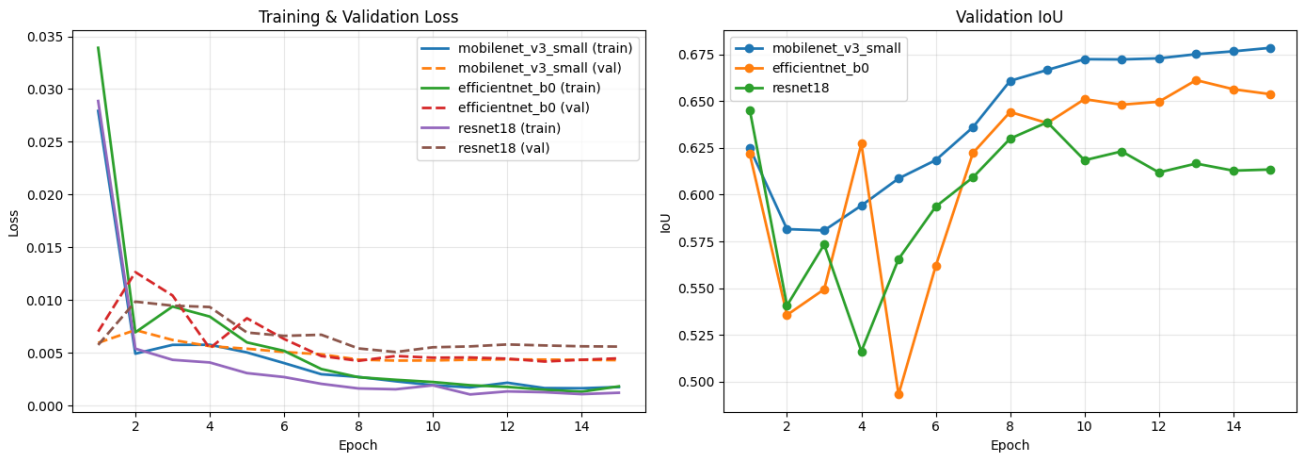
    for imgs, bboxes in loader:
        imgs = imgs.to(device)
        bboxes = bboxes.to(device)

        preds = model(imgs)
        loss = criterion(preds, bboxes)
        total_loss += loss.item() * imgs.size(0)

        for i in range(len(bboxes)):
            iou = compute_iou(bboxes[i].cpu().numpy(), preds[i].cpu().numpy())
            total_iou += iou
            count += 1

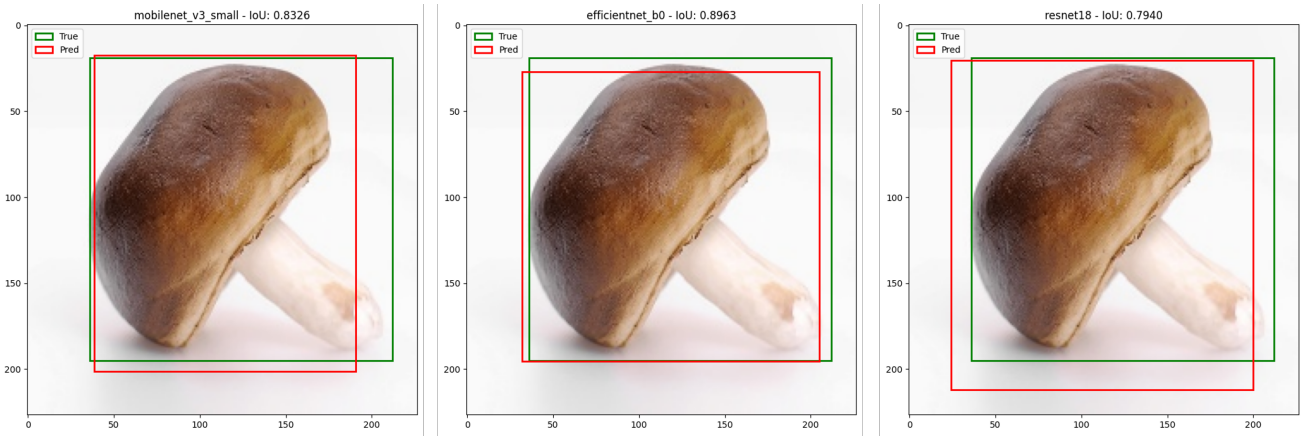
    avg_loss = total_loss / len(loader.dataset)
    avg_iou = total_iou / count if count > 0 else 0.0
    return avg_loss, avg_iou
```

Ở bước này, ta huấn luyện mô hình theo tiêu chuẩn *hồi quy* (regression) và đánh giá bằng *IoU* (Intersection-over-Union). Hàm mất mát (loss function) sử dụng `SmoothL1Loss` (SmoothL1/Huber loss), một lựa chọn phổ biến cho bbox regression do cân bằng giữa L1 và L2, giúp ổn định khi có nhiễu (noise) hoặc ngoại lai (outliers). Chỉ số IoU được tính trên bounding box chuẩn hoá và phản ánh mức độ chồng lấp hình học (geometric overlap) giữa dự đoán và nhãn.



Hình 8: Biểu đồ loss và IoU sau khi train xong model.

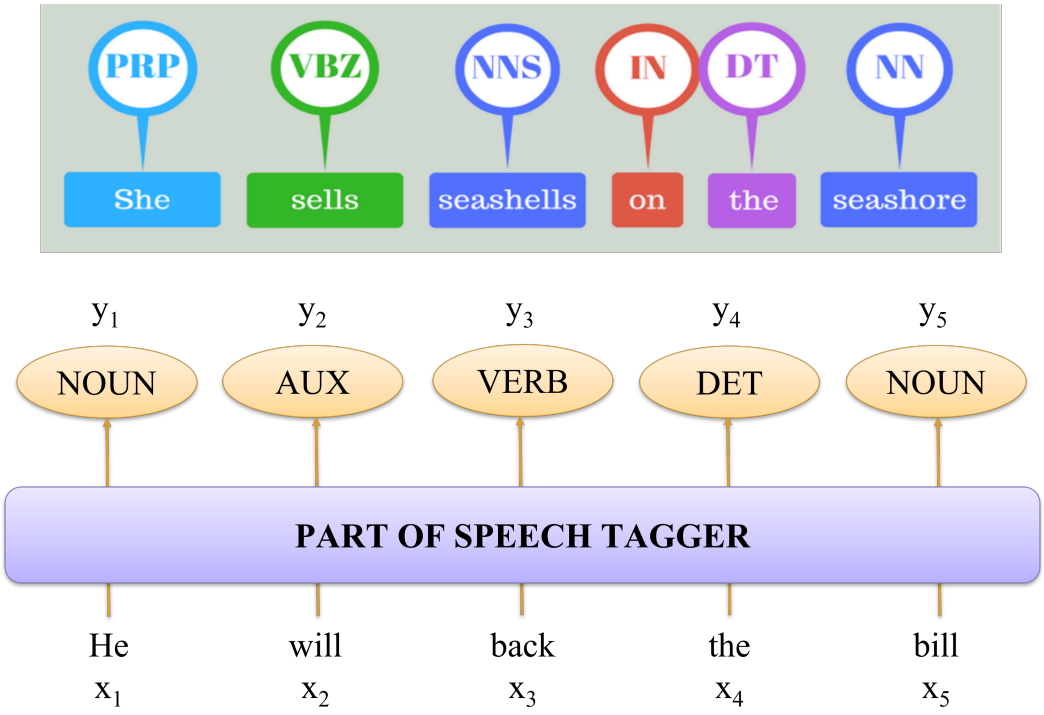
Loss (loss) giảm nhanh trong vài epoch đầu rồi hội tụ (convergence); các đường validation (validation) ổn định quanh mức thấp. Về chất lượng định vị, IoU (Intersection-over-Union, IoU) tăng dần theo epoch và MobileNetV3-Small đạt IoU cao nhất, EfficientNet-B0 đứng thứ hai (dao động mạnh giai đoạn đầu), còn ResNet18 thấp nhất.



Hình 9: Kết quả inference của ba model vừa train được.

V. Bài toán POS Tagging tiếng Việt với PhoBERT

Trong phần này, chúng ta thực hành bài toán *gán nhãn từ loại* (Part-of-Speech tagging, POS tagging) theo hướng *gán nhãn chuỗi* (sequence labeling). Mục tiêu là fine-tune *mô hình ngôn ngữ tiền huấn luyện* (pretrained language model) PhoBERT để dự đoán nhãn POS cho từng token trong câu, sau đó đánh giá mô hình bằng các chỉ số như *độ chính xác* (accuracy) và *F1-score*.



Hình 10: Minh hoạ POS tagging: gán nhãn từ loại cho từng token dựa trên ngữ cảnh câu.

Bước 1: Tải dữ liệu, import thư viện

```
!gdown 13Tjq5dccZguiPa0pTSrl3oKKWEiITmWf
!unzip -o pos.zip -d pos_data

import os
import re
import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from typing import List, Dict, Tuple, Set
```

```

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import AutoModel, AutoTokenizer
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder

TRAIN_DATASET = "pos_data/vi_train.txt"
TEST_DATASET = "pos_data/vi_test.txt"
PREPROC_TRAIN_DATASET = "vi_train_preproc.jsonl"
PREPROC_TEST_DATASET = "vi_test_preproc.jsonl"

PHOBERT_MODEL = "vinai/phobert-base"
MAX_LENGTH = 128
BATCH_SIZE = 64
LEARNING_RATE = 2e-5
EPOCHS = 15
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")
```

Ở bước này, ta tải dữ liệu POS và import các thư viện cần thiết cho bài toán POS tagging. PyTorch được dùng để xây dựng mô hình (model) và huấn luyện (training), trong khi Transformers cung cấp PhoBERT và tokenizer. Ta thiết lập các siêu tham số (hyperparameters) như `MAX_LENGTH`, `BATCH_SIZE`, `LEARNING_RATE` và số epoch.

Bước 2: Tiền xử lý dữ liệu

```
VALID_TAGS = {
    'A', 'C', 'E', 'L', 'M', 'N', 'Nc', 'Np', 'Nu',
    'P', 'PUNCT', 'R', 'S', 'T', 'UNKNOWN_WORD', 'V', 'X'
}

PUNCT = {
    "‘‘", "’’", '""', ".", "...", "", """, """, "!", '"', "#", "$", "%", "&",
    "’", "(", ")", "*", "+", ",", "-", "/", ":", ";", "<", "=", ">", "?",
    "@", "[", "\\", "]", "_", "‘", "{", "|", "}", "~", "’", ".)"
}

UNKNOWN_WORDS = {'p', 'v', 'm', 'a', 'Ny', 'Ap', 'Vy', 'Vb', 'Ab', 'NC', 'Eb', 'H'}

def fix_sentence(sentence: str) -> List[str]:
    sentence = re.sub(r'/+', '/', sentence)
    sentence = re.sub(r'(-/-)(\d+/\w+)', r'\1 \2', sentence)
    sentence = re.sub(r'(\d+/\w+)(-/-)', r'\1 \2', sentence)
    sentence = re.sub(r'\s+', ' ', sentence).strip()

    tokens = sentence.split()
    clean_tokens = []
```

```
    for tok in tokens:
        matches = re.findall(r'([^\s/]+)/([^\s/]+)', tok)
        if matches:
            for w, t in matches:
                clean_tokens.append(f"{w}/{t}")
    return clean_tokens

def split_slash(tok: str) -> Tuple[str, str]:
    if '/' in tok:
        word, tag = tok.rsplit('/', 1)
    else:
        word, tag = tok, "X"

    word = word.lower().replace("_", " ")
    if word in PUNCT or tag in PUNCT:
        tag = "PUNCT"
    elif tag in UNKNOWN_WORDS:
        tag = "UNKNOWN_WORD"
    elif tag not in VALID_TAGS:
        tag = "X"
    return word, tag

def preprocess_data(filename: str) -> List[Tuple[List[str], List[str]]]:
    words_tags_list = []
    with open(filename, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            clean_tokens = fix_sentence(line)
            if not clean_tokens:
                continue

            words, tags = [], []
            for tok in clean_tokens:
                word, tag = split_slash(tok)
                words.append(word)
                tags.append(tag)
            words_tags_list.append((words, tags))
    return words_tags_list

def save_preprocessed_dataset(output_filename: str, words_tags):
    with open(output_filename, "w", encoding="utf-8") as f:
        for words, tags in words_tags:
            f.write(json.dumps({"tokens": words, "tags": tags}, ensure_ascii=False) + "
                                                \n")

train_wt_list = preprocess_data(TRAIN_DATASET)
save_preprocessed_dataset(PREPROC_TRAIN_DATASET, train_wt_list)

test_wt_list = preprocess_data(TEST_DATASET)
save_preprocessed_dataset(PREPROC_TEST_DATASET, test_wt_list)
```

Ở bước này, dữ liệu được chuẩn hoá về định dạng `tokens/tags` để tiện huấn luyện. Ta xử lý các trường hợp dấu câu (punctuation), nhãn lạ (unknown tag), và các nhãn ngoài tập nhãn hợp lệ. Kết quả được lưu dạng JSONL để tải nhanh ở các bước sau.

Bước 3: Tải dữ liệu tiền xử lý và mã hoá nhãn

```python
def load_data(file_path: str) -> List[Dict]:
    data = []
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if line:
                data.append(json.loads(line))
    return data

train_data = load_data(PREPROC_TRAIN_DATASET)
test_data = load_data(PREPROC_TEST_DATASET)

all_tags = set()
for sample in train_data + test_data:
    all_tags.update(sample["tags"])

label_encoder = LabelEncoder()
label_encoder.fit(sorted(all_tags))
num_labels = len(label_encoder.classes_)
print(f"Number of unique tags: {num_labels}")
```

Ở bước này, ta ánh xạ nhãn POS sang chỉ số (index) bằng *bộ mã hoá nhãn* (label encoder). Việc mã hoá giúp mô hình học dưới dạng phân loại (classification) trên tập nhãn rời rạc.

Bước 4: Dataset cho POS tagging

```python
class POSDataset(Dataset):
    def __init__(self, data: List[Dict], tokenizer, label_encoder, max_length: int =
                                                128):
        self.data = data
        self.tokenizer = tokenizer
        self.label_encoder = label_encoder
        self.max_length = max_length
        self.pad_token_id = tokenizer.pad_token_id if tokenizer.pad_token_id is not
                                                None else tokenizer.eos_token_id

    def _tokenize_with_alignment(self, tokens: List[str], tags: List[str]):
        input_ids = [self.tokenizer.bos_token_id]
        label_ids = [-100]

        for word, tag in zip(tokens, tags):
```

```
            word_pieces = self.tokenizer.tokenize(word)
            if not word_pieces:
                word_pieces = [self.tokenizer.unk_token]
            word_piece_ids = self.tokenizer.convert_tokens_to_ids(word_pieces)

            if len(input_ids) + len(word_piece_ids) + 1 > self.max_length:
                break

            input_ids.extend(word_piece_ids)
            tag_id = self.label_encoder.transform([tag])[0]
            label_ids.extend([tag_id] + [-100] * (len(word_piece_ids) - 1))

        input_ids.append(self.tokenizer.eos_token_id)
        label_ids.append(-100)

        attention_mask = [1] * len(input_ids)
        pad_len = self.max_length - len(input_ids)
        if pad_len > 0:
            input_ids.extend([self.pad_token_id] * pad_len)
            attention_mask.extend([0] * pad_len)
            label_ids.extend([-100] * pad_len)
        else:
            input_ids = input_ids[:self.max_length]
            attention_mask = attention_mask[:self.max_length]
            label_ids = label_ids[:self.max_length]
            input_ids[-1] = self.tokenizer.eos_token_id
            attention_mask[-1] = 1
            label_ids[-1] = -100

        return input_ids, attention_mask, label_ids

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        input_ids, attention_mask, label_ids = self._tokenize_with_alignment(
            sample["tokens"], sample["tags"]
        )
        return {
            "input_ids": torch.tensor(input_ids, dtype=torch.long),
            "attention_mask": torch.tensor(attention_mask, dtype=torch.long),
            "labels": torch.tensor(label_ids, dtype=torch.long),
        }
```

Ở bước này, ta xây dựng *dataset* (dataset) cho POS tagging với PhoBERT. Vì tokenizer của PhoBERT sinh *từ con* (subword), ta cần *căn chỉnh nhãn* (label alignment) theo subword: nhãn POS chỉ gán cho subword đầu tiên của mỗi từ, còn các subword còn lại dùng giá trị `-100` để bỏ qua khi tính loss (ignore index). Cách làm này giúp huấn luyện đúng bản chất gán nhãn theo từ, nhưng vẫn tận dụng tokenization theo subword.

Bước 5: Khai báo mô hình PhoBERTForPOS – Bài tập điền code

Code Exercise

```python
class PhoBERTForPOS(nn.Module):
    def __init__(self, model_name: str, num_labels: int, dropout: float = 0.1):
        super(PhoBERTForPOS, self).__init__()
        self.phobert = ***Your code here***
        self.dropout = nn.Dropout(dropout)
        self.classifier = ***Your code here***

    def forward(self, input_ids, attention_mask):
        outputs = ***Your code here***
        sequence_output = ***Your code here***
        sequence_output = self.dropout(sequence_output)
        logits = ***Your code here***
        return logits
```

Ở bài tập này, bạn cần hoàn thiện mô hình PhoBERT cho POS tagging bằng cách: (1) khởi tạo *PhoBERT encoder* (PhoBERT encoder) từ `AutoModel.from_pretrained`, (2) tạo *lớp phân loại* (classification layer) ánh xạ từ *ẩn trạng thái* (hidden size) sang `num_labels`, và (3) triển khai forward theo luồng *token-level classification* (token-level classification). Đầu ra `logits` có kích thước $[B, L, |\mathcal{Y}|]$ với B là batch size, L là độ dài chuỗi (sequence length), và $|\mathcal{Y}|$ là số nhãn.

Bước 6: Khởi tạo tokenizer, tạo DataLoader

```python
tokenizer = AutoTokenizer.from_pretrained(PHOBERT_MODEL, use_fast=False)
model = PhoBERTForPOS(PHOBERT_MODEL, num_labels).to(DEVICE)

train_size = int(0.9 * len(train_data))
train_subset = train_data[:train_size]
val_subset = train_data[train_size:]

train_dataset = POSDataset(train_subset, tokenizer, label_encoder, MAX_LENGTH)
val_dataset = POSDataset(val_subset, tokenizer, label_encoder, MAX_LENGTH)
test_dataset = POSDataset(test_data, tokenizer, label_encoder, MAX_LENGTH)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

Ở bước này, ta khởi tạo tokenizer và mô hình PhoBERT, sau đó tạo tập train/validation/test. DataLoader hỗ trợ tạo mini-batch và xáo trộn (shuffle) ở train để tối ưu ổn định hơn.

Bước 7: Hàm huấn luyện và đánh giá

```python
def train_epoch(model, dataloader, optimizer, criterion, device):
    model.train()
    total_loss, total_correct, total_tokens = 0, 0, 0

    pbar = tqdm(dataloader, desc="Training")
    for batch in pbar:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        optimizer.zero_grad()
        logits = model(input_ids, attention_mask)

        loss = criterion(logits.view(-1, num_labels), labels.view(-1))
        loss.backward()
        optimizer.step()

        predictions = torch.argmax(logits, dim=-1)
        mask = labels != -100
        total_correct += ((predictions == labels) & mask).sum().item()
        total_tokens += mask.sum().item()
        total_loss += loss.item()

        pbar.set_postfix({"loss": f"{loss.item():.4f}"})

    avg_loss = total_loss / len(dataloader)
    accuracy = total_correct / total_tokens if total_tokens > 0 else 0
    return avg_loss, accuracy

@torch.no_grad()
def evaluate(model, dataloader, criterion, device):
    model.eval()
    total_loss, total_correct, total_tokens = 0, 0, 0
    all_predictions, all_labels = [], []

    for batch in tqdm(dataloader, desc="Evaluating"):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        logits = model(input_ids, attention_mask)
        loss = criterion(logits.view(-1, num_labels), labels.view(-1))

        predictions = torch.argmax(logits, dim=-1)
        mask = labels != -100

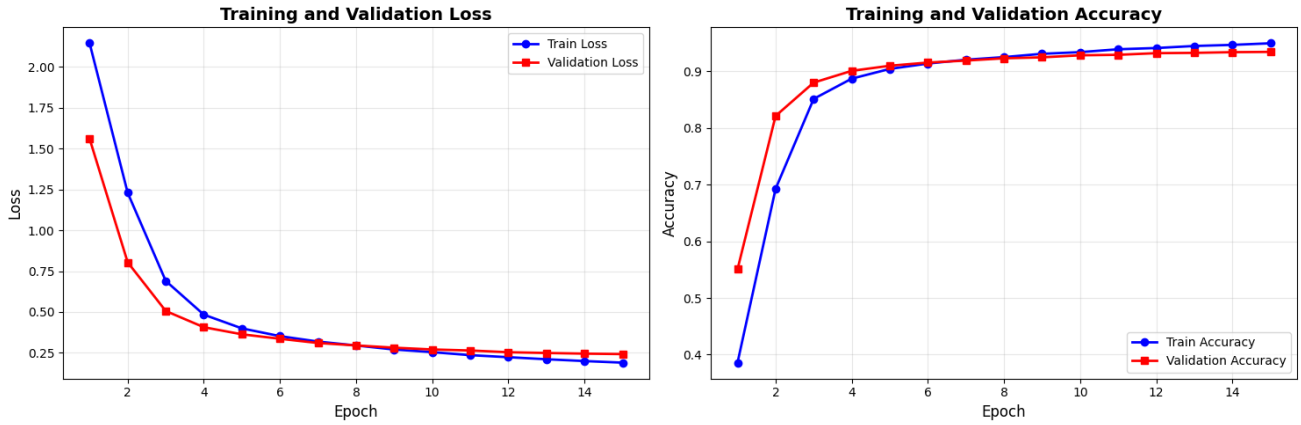
        all_predictions.extend(predictions[mask].cpu().numpy())
        all_labels.extend(labels[mask].cpu().numpy())

        total_correct += ((predictions == labels) & mask).sum().item()
        total_tokens += mask.sum().item()
```

```
        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    accuracy = total_correct / total_tokens if total_tokens > 0 else 0
    return avg_loss, accuracy, all_predictions, all_labels
```

Ở bước này, loss dùng *cross-entropy* (cross-entropy) với `ignore_index=-100` để bỏ qua các vị trí subword không mang nhãn. Accuracy được tính trên các token có nhãn hợp lệ (mask).



Hình 11: Biểu đồ Loss và Accuracy của training và validation.

VI. Câu hỏi trắc nghiệm

1. (Tính toán – Kích thước tensor theo pipeline (tensor shapes)) Xét pipeline *backbone CNN tiền huấn luyện* (pretrained CNN backbone) + *đầu hồi quy* (regression head). Để tránh tràn lề, ta viết pipeline theo 2 dòng:

$$\mathbf{x} \in \mathbb{R}^{B\times 3\times 320\times 320} \to \text{Backbone} \to \mathbf{F} \in \mathbb{R}^{B\times C\times H\times W} \to \text{AdaptiveAvgPool2d(1)}$$

$$\to \text{Flatten} \to \text{FC} \to \hat{\mathbf{b}} \in \mathbb{R}^{B\times 4}.$$

Giả sử dùng MobileNetV3-Small, số kênh cuối (final channels) $C = 576$ và tổng stride (total stride) bằng 32. Với $B = 8$, bộ kích thước đúng nhất của ($\mathbf{F}$, Sau Pool, Sau Flatten, $\hat{\mathbf{b}}$) là:

A. $(8, 576, 20, 20),\ (8, 576, 1, 1),\ (8, 576),\ (8, 4)$

B. $(8, 576, 10, 10),\ (8, 576, 10, 10),\ (8, 576),\ (8, 4)$

C. $(8, 576, 10, 10),\ (8, 576, 1, 1),\ (8, 576),\ (8, 4)$

D. $(8, 576, 10, 10),\ (8, 576, 1, 1),\ (8, 256),\ (8, 4)$

2. (Tính toán – IoU (Intersection-over-Union, IoU) với bbox chuẩn hoá (normalized bbox)) Cho *bbox nhãn* (ground-truth bbox) và *bbox dự đoán* (predicted bbox) ở dạng chuẩn hoá:

$$\mathbf{b}_{gt} = (0.10, 0.20, 0.60, 0.70), \qquad \mathbf{b}_{pr} = (0.30, 0.10, 0.80, 0.50).$$

Với:

$$\text{IoU} = \frac{A_\cap}{A_{gt} + A_{pr} - A_\cap},$$

giá trị IoU xấp xỉ là:

A. 0.125

B. 0.200

C. 0.250

D. 0.360

3. (Tính toán – SmoothL1Loss (Smooth L1 loss) cho vector bbox) Dùng *SmoothL1Loss* (Smooth L1 loss) với $\beta = 1$ và mean theo 4 toạ độ:

$$\ell(d) = \begin{cases} \frac{1}{2}d^2 & \text{nếu } |d| < 1, \\ |d| - \frac{1}{2} & \text{nếu } |d| \geq 1. \end{cases}$$

Cho:

$$\mathbf{b}_{gt} = (0.10, 0.20, 0.60, 0.70), \qquad \mathbf{b}_{pr} = (0.18, 0.10, 0.55, 0.90).$$

Loss trung bình xấp xỉ là:

A. 7.36×10^{-4}

B. 7.36×10^{-3}

C. 2.95×10^{-2}

D. 7.36×10^{-2}

4. (Tính toán – Số tham số (parameter count) của head hồi quy (regression head)) Head:

$$\mathrm{FC}_1 : \mathbb{R}^{1280} \to \mathbb{R}^{256}, \qquad \mathrm{FC}_2 : \mathbb{R}^{256} \to \mathbb{R}^4,$$

mỗi linear có bias. Tổng số tham số trainable (trainable parameters) là:

A. 328,704

B. 328,964

C. 330,244

D. 331,524

5. (Tính toán – Chuyển bbox chuẩn hoá sang pixel và IoU (IoU invariance))

Ảnh có $W = 640,\ H = 480$. Cho:

$$\mathbf{b}_{gt} = (0.25, 0.10, 0.75, 0.60), \qquad \mathbf{b}_{pr} = (0.30, 0.05, 0.90, 0.70).$$

Chuyển sang pixel:

$$(x_{\min}, y_{\min}, x_{\max}, y_{\max}) = (x_{\min}W,\ y_{\min}H,\ x_{\max}W,\ y_{\max}H).$$

IoU xấp xỉ là:

A. 0.542

B. 0.610

C. 0.421

D. 0.750

6. (Tính toán – Truncation (truncation) và căn chỉnh nhãn (subword alignment)) Trong căn chỉnh nhãn theo subword (subword alignment), khởi tạo:

$$\text{input_ids} = [\text{BOS}], \quad \text{label_ids} = [-100].$$

Với một từ có k subword, label chỉ gán cho subword đầu (first subword): $[\text{tag_id}, -100, \dots]$. Điều kiện dừng:

$$\text{Nếu } |\text{input_ids}| + k + 1 > L \text{ thì break.}$$

Cho $L = 12$ và số subword theo từng từ lần lượt là $(1, 2, 3, 1, 4)$. Hỏi: (i) giữ được bao nhiêu từ, và (ii) có bao nhiêu label hợp lệ (khác -100)?

A. (4, 4)

B. (5, 5)

C. (4, 7)

D. (3, 3)

7. (Tính toán – Kích thước logits (logits shape) khi tính loss) Cho $B = 64$, $L = 128$, số nhãn $|\mathcal{Y}| = 17$. Logits:

$$\mathbf{Z} \in \mathbb{R}^{B \times L \times |\mathcal{Y}|}.$$

Tính loss bằng:

$$\mathrm{CE}\Big(\mathbf{Z}.view(-1, |\mathcal{Y}|),\ \mathbf{y}.view(-1)\Big).$$

Kích thước đúng là:

A. $(64, 128, 17)$ và $(64, 128)$

B. $(8192, 17)$ và (8192)

C. $(64, 17, 128)$ và $(64, 128)$

D. $(128, 64, 17)$ và $(128, 64)$

8. (Tính toán – Cross-entropy với ignore_index (ignore_index)) Với $|\mathcal{Y}| = 3$, có 2 token; token 2 có label -100 nên bị bỏ qua (ignored). Cho:

$$\mathbf{z}_1 = (2, 0, -1),\ y_1 = 0; \qquad \mathbf{z}_2 = (0, 1, 0),\ y_2 = -100.$$

Loss mean xấp xỉ là:

A. 0.171

B. 0.693

C. 1.171

D. 2.171

9. (Tính toán – Accuracy theo mask (mask) loại bỏ -100) Cho:

$$\hat{\mathbf{y}} = (0, 1, 2, 1, 0), \qquad \mathbf{y} = (0, -100, 2, 0, 0).$$

Accuracy trên các vị trí có $\mathbf{y} \neq -100$ là:

A. 0.50

B. 0.75

C. 0.80

D. 1.00

10. (Tính toán – Inference: ánh xạ subword về token gốc (word_map)) Trong suy luận (inference), chỉ **subword đầu** (first subword) của mỗi từ được dùng để lấy nhãn POS, các subword còn lại bỏ qua. Cho:

$$\texttt{word_map} = [\texttt{None}, 0, 1, \texttt{None}, 2, \texttt{None}, \texttt{None}, \texttt{None}],$$

predicted label ids:

$$[99,\ 3,\ 5,\ 2,\ 1,\ 4,\ 7,\ 8],$$

và label_encoder (label encoder):

$$1 \mapsto \texttt{V}, \quad 3 \mapsto \texttt{P}, \quad 5 \mapsto \texttt{R}.$$

Kết quả POS cuối cùng theo token gốc là:

A. `[("tôi","P"), ("đang","R"), ("học","V")]`

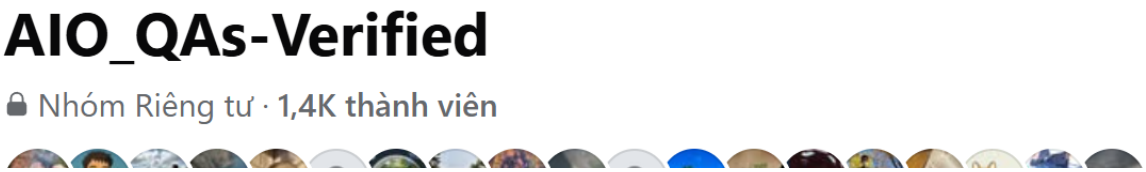
B. `[("tôi","R"), ("đang","P"), ("học","V")]`

C. `[("tôi","P"), ("đang","V"), ("học","R")]`

D. `[("tôi","V"), ("đang","R"), ("học","P")]`

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải tại đây.
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây (Lưu ý: Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.



Hình 12: Hình ảnh group facebook AIO Q&A

4. **Rubric:**

Phần	Kiến Thức	Đánh Giá
1	- Hiểu bài toán định vị một đối tượng (single-object localization) và hồi quy khung giới hạn (bounding box regression). - Nắm các dạng biểu diễn bbox: $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ và chuẩn hoá (normalization) về $[0, 1]$.	- Giải thích được vì sao thiết lập này không cần nhãn lớp (class label) hay điểm tin cậy (objectness/confidence). - Tính và diễn giải được IoU (Intersection-over-Union, IoU) cho một cặp bbox dự đoán/nhãn.
2	- Hiểu POS tagging (Part-of-Speech tagging, POS) như bài toán gán nhãn chuỗi (sequence labeling). - Nắm ý nghĩa biểu diễn theo ngữ cảnh (contextualized representation) khi fine-tune PhoBERT (pretrained language model).	- Giải thích được mục tiêu tối ưu bằng cross-entropy (cross-entropy) theo token và vai trò ignore index (ignore_index) cho subword. - Đọc và diễn giải được classification report (precision/recall/F1) cho các nhãn POS chính.
3	- Hiểu sự khác nhau giữa token-level POS và căn chỉnh nhãn theo subword (subword alignment) khi dùng tokenizer BPE. - Nắm pipeline tổng quát: Tokenization → Encoder (PhoBERT) → Linear head → Tag prediction.	- Phân tích được các lỗi thường gặp: nhầm dấu câu (punctuation), nhãn hiếm (rare tags), và sai lệch do tách từ (tokenization). - Đánh giá được tác động của độ dài chuỗi (max length) tới hiệu năng và tốc độ.